

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ ПО РАЗРАБОТКЕ СИСТЕМЫ УПРАВЛЕНИЯ АВТОЗАПРАВОЧНОЙ СТАНЦИИ НА PYTHON

по дисциплине
«Информатика и программирование»

Студент		
гр. БИН-25-3	_____	А. Е. Филатов
Ассистент		
преподавателя	_____	М.В. Водяницкий

Содержание

1	Введение	3
1.1	Принципы Проектирования.....	3
2	Выполнение работы.....	4
2.1	Архитектура программы	4
2.2	Импорт библиотек.....	4
2.3	Класс FuelType	4
2.4	Класс Tank	4
2.5	Класс Nozzle	6
2.6	Класс Pump	6
2.7	Класс Operation.....	7
2.8	Класс GasStation.....	7
2.9	Класс Pump	8
2.10	Метод init default setup.....	8
2.11	Сохранение и загрузка состояния	9
2.12	Вспомогательные методы.....	10
2.13	Метод serve customer	11
2.14	check tanks	12
2.15	refill tank	12
2.16	show balance	12
2.17	transfer fuel.....	13
2.18	manage tanks.....	13
2.19	show pumps	14
2.20	show warning	14
2.21	run	14
2.22	start program	15
3	Тестирование	16
4	Заключение	17

1 Введение

Автоматизация процессов на автозаправочных станциях (АЗС) является важным аспектом современного управления топливными ресурсами и обслуживания клиентов. В данном отчете представлена разработка системы управления АЗС на языке программирования Python. Целью данной работы является создание эффективного и надежного программного обеспечения, которое позволит оптимизировать процессы продажи топлива, учета запасов и взаимодействия с клиентами.

Целью данной лабораторной работы является разработка системы управления АЗС, полностью соответствующая техническому заданию с упором на надежность данной системы и удобство использования для конечного пользователя.

1.1 Принципы Проектирования

При разработке были заложены следующие принципы проектирования:

- Модульность: Система была разработана с использованием модульного подхода, что позволяет легко добавлять новые функции и изменять существующие без влияния на другие части системы.
- Пользовательский интерфейс: Особое внимание было уделено созданию интуитивно понятного интерфейса для операторов АЗС, что способствует быстрому обучению и снижению ошибок при эксплуатации.
- Надежность: В систему были встроены механизмы обработки ошибок и резервного копирования данных для обеспечения непрерывной работы АЗС.
- Производительность: Оптимизация кода и использование эффективных алгоритмов позволили обеспечить высокую скорость обработки транзакций и управления запасами топлива для удобства пользования.
- Безопасность: Внедрение мер безопасности для защиты данных клиентов и предотвращения несанкционированного доступа к системе.

Данные принципы проектирования обеспечивают создание системы управления АЗС, которая отвечает современным требованиям и способствует эффективной работе автозаправочных станций.

2 Выполнение работы

2.1 Архитектура программы

Архитектура представленной программы модульная и объектно-ориентированная, с чётким разделением ответственности между компонентами. Основные компоненты:

- Это классы, описывающие сущности АЗС и их поведение;
- Основной контроллер "GasStation";
- Система сохранения состояния "gas station data.json";
- Безопасность и ограничения "min level" "max level" и "can supply".

2.2 Импорт библиотек

В начале программы импортируются необходимые модули и библиотеки, которые обеспечивают функциональность и поддержку различных операций. На рисунке 1 представлен код программы.

```
1 import json
2 import os
3 from datetime import datetime
4 from typing import List, Dict, Optional
```

Рисунок 1 – Листинг программы для importModules

- json - для сохранения/загрузки данных в JSON-формате;
- os - для работы с операционной системой (очистка экрана, проверка файлов);
- datetime - для получения текущей даты и времени;
- typing - для аннотации типов (улучшает читаемость кода).

Это необходимо для обеспечения функциональности программы и взаимодействия с внешними ресурсами.

2.3 Класс FuelType

Класс, содержащий строки с названиями видов топлива: "АИ-92" "АИ-95" "АИ-98" "ДТ". На рисунке 2 представлен код программы.

```
1 class FuelType:
2     AI92 = "АИ-92"
3     AI95 = "АИ-95"
4     AI98 = "АИ-98"
5     DT = "ДТ"
```

Рисунок 2 – Листинг программы для FuelType

Централизует символические имена топлива, уменьшает риск опечаток. Плюс данной константы, это- изменил в одном месте, то изменилось повсеместно.

2.4 Класс Tank

Моделирует резервуар для хранения топлива. Хранит информацию о типе топлива, идентификаторе, максимальном и текущем объеме, минимальном допустимом уровне и состоянии (включена/отключена). На рисунке 3 представлен код программы.

```

1 class Tank:
2     def __init__(self, fuel_type: str, tank_id: int,
3         max_volume: int, min_level: int = 2000):
4         self.fuel_type = fuel_type
5         self.tank_id = tank_id
6         self.max_volume = max_volume
7         self.current_volume = max_volume # начинаем с
8         self.min_level = min_level
9         self.enabled = True
10
11     def is_low(self) -> bool:
12         return self.current_volume < self.min_level
13
14     def disable_if_low(self):
15         if self.is_low():
16             self.enabled = False
17
18     def can_supply(self, liters: int) -> bool:
19         return self.enabled and self.current_volume >=
20         liters
21
22     def withdraw(self, liters: int):
23         if self.can_supply(liters):
24             self.current_volume -= liters
25             self.disable_if_low()
26         else:
27             raise ValueError("Недостаточно топлива или
28             цистерна отключена")
29
30     def refill(self, liters: int):
31         if self.current_volume + liters > self.max_volume:
32             raise ValueError("Превышение максимального объема")
33         self.current_volume += liters
34
35     def to_dict(self):
36         return {
37             "fuel_type": self.fuel_type,
38             "tank_id": self.tank_id,
39             "max_volume": self.max_volume,
40             "current_volume": self.current_volume,
41             "min_level": self.min_level,
42             "enabled": self.enabled
43         }
44
45 ...

```

Рисунок 3 – Листинг программы для Tank

- is low() — проверяет, упал ли уровень ниже порогового;
- disable if low() — автоматически отключает цистерну при низком уровне;
- can supply(), withdraw() — обеспечивают безопасную выдачу топлива;
- refill() — пополнение топлива с проверкой на переполнение;
- to dict() и from dict() — преобразования или преобразование объекта для сохранения состояния.

Очень важный блок: модель «реальной» цистерны, с проверками безопасности (нет выдачи если мало топлива, нет переполнения при пополнении).

2.5 Класс Nozzle

Связывает конкретный вид топлива с конкретной цистерной для колонки (помогает представить пистолет на колонке), а `is available()` говорит, можно ли из этого сопла раздавать (если связанная цистерна включена). На рисунке 4 представлен код программы.

```

1 class Nozzle:
2     def __init__(self, fuel_type: str, tank: Tank):
3         self.fuel_type = fuel_type
4         self.tank = tank
5
6     def is_available(self) -> bool:
7         return self.tank.enabled
8
9     def get_tank_info(self) -> str:
10        return f"{self.fuel_type} №{self.tank.tank_id}"

```

Рисунок 4 – Листинг программы для Nozzle

- Связывает конкретный вид топлива с конкретной цистерной для колонки (помогает представить пистолет на колонке);
- `is available()` говорит, можно ли из этого сопла раздавать (если связанная цистерна включена).

Отделяет логику представления топлива на колонке от самой цистерны. Упрощает работу с разными соплами на одной колонке.

2.6 Класс Pump

Группирует сопла в одну колонку с идентификатором. Возвращает все сопла или только доступные (если цистерна включена). На рисунке 8 представлен код программы.

```

1 class Pump:
2     def __init__(self, pump_id: int, nozzles: List[Nozzle]):
3         self.pump_id = pump_id
4         self.nozzles = nozzles
5
6     def get_available_fuels(self) -> List[Nozzle]:
7         return [n for n in self.nozzles if n.is_available()]
8
9     def get_all_fuels(self) -> List[Nozzle]:
10        return self.nozzles
11
12    def __str__(self):
13        fuels = ", ".join([n.fuel_type for n in self.nozzles
14        ])
15        return f"Колонка {self.pump_id}: {fuels}"

```

Рисунок 5 – Листинг программы для Pump

- Упрощает меню выбора колонки и отображение, какие топлива доступны;
- Логическая модель «колонки» соответствует реальному АЗС.

Нужен для структуры станции: колонки состоят из сопел, и на уровне Pump удобно работать с интерфейсом.

2.7 Класс Operation

Логирует каждое действие на АЗС: продажу, пополнение, аварию и т.д. На рисунке 6 представлен код программы.

```

1 class Operation:
2     def __init__(self, op_type: str, details: str, timestamp
      : str = None):
3         self.op_type = op_type
4         self.details = details
5         self.timestamp = timestamp or datetime.now().
      strftime("%Y-%m-%d %H:%M:%S")
6
7     def to_dict(self):
8         return {"op_type": self.op_type, "details": self.
      details, "timestamp": self.timestamp}
9
10    @classmethod
11    def from_dict(cls, data):
12        return cls(data["op_type"], data["details"], data["
      timestamp"])
13
14    def __str__(self):
15        return f"[{self.timestamp}] {self.op_type}: {self.
      details}"

```

Рисунок 6 – Листинг программы для Operation

Модуль логирования для понимания, что происходило в системе.

2.8 Класс GasStation

Это центральный класс, объединяющий все компоненты и реализующий пользовательский интерфейс. На рисунке 7 представлен код программы.

```

1 class GasStation:
2     FUEL_PRICES = {
3         FuelType.AI92: 57.50,
4         FuelType.AI95: 58.30,
5         FuelType.AI98: 61.20,
6         FuelType.DT: 54.80
7     }
8
9     def __init__(self):
10        self.tanks: List[Tank] = []
11        self.pumps: List[Pump] = []
12        self.operations: List[Operation] = []
13        self.stats = {
14            "total_income": 0.0,
15            "cars_served": 0,
16            "fuel_sold": {ft: 0 for ft in [FuelType.AI92,
      FuelType.AI95, FuelType.AI98, FuelType.DT]},
17            "fuel_income": {ft: 0.0 for ft in [FuelType.AI92
      , FuelType.AI95, FuelType.AI98, FuelType.DT]}
18        }
19        self.emergency_mode = False
20        self._init_default_setup()

```

Рисунок 7 – Листинг программы для GasStation

- Инициализация (init default setup)- Создаёт 5 цистерн (включая две для АИ-95). Настраивает 8 колонок с разным набором топлива. Реализует логику распределения: колонки 14 используют первую цистерну АИ-95, 5-8- вторую.

- Сохранение и загрузка данных. Все данные (цистерны, операции, статистика) сохраняются в gas station data.json. При загрузке восстанавливается не только состояние, но и связи между колонками и цистернами.

- Основные функции меню: обслужить клиента, проверка цистерн, пополнение топлива, баланс и статистика, история операций, перекачка топлива, управление цистернами, аварийный режим.

Сердце программы: дальше идут методы, которые реализуют меню и действия.

2.9 Класс Pump

Группирует сопла в одну колонку с идентификатором. Возвращает все сопла или только доступные (если цистерна включена). На рисунке 8 представлен код программы.

```

1 class Pump:
2     def __init__(self, pump_id: int, nozzles: List[Nozzle]):
3         self.pump_id = pump_id
4         self.nozzles = nozzles
5
6     def get_available_fuels(self) -> List[Nozzle]:
7         return [n for n in self.nozzles if n.is_available()]
8
9     def get_all_fuels(self) -> List[Nozzle]:
10        return self.nozzles
11
12    def __str__(self):
13        fuels = ", ".join([n.fuel_type for n in self.nozzles
14        ])
15        return f"Колонка {self.pump_id}: {fuels}"

```

Рисунок 8 – Листинг программы для Pump

- Упрощает меню выбора колонки и отображение, какие топлива доступны;
- Логическая модель «колонок» соответствует реальному АЗС.

Нужен для структуры станции: колонки состоят из сопел, и на уровне Pump удобно работать с интерфейсом.

2.10 Метод init default setup

Даёт реалистичную схему подключения колонок и цистерн. На рисунке 9 представлен код программы.


```

1 def _init_default_setup(self):
2     # Создаем цистерны
3     self.tanks = [
4         Tank(FuelType.AI92, 1, 20000),
5         Tank(FuelType.AI95, 1, 20000),
6         Tank(FuelType.AI95, 2, 20000),
7         Tank(FuelType.AI98, 1, 15000),
8         Tank(FuelType.DT, 1, 25000)
9     ]
10
11     # Связываем колонки с цистернами
12     tank_map = {(t.fuel_type, t.tank_id): t for t in self.tanks}
13
14     # Схема подключения
15     pump_config = {
16         1: [FuelType.AI92, FuelType.AI95],
17         2: [FuelType.AI92, FuelType.AI95],
18         3: [FuelType.AI92, FuelType.AI95, FuelType.AI98,
19            FuelType.DT],
20         4: [FuelType.AI92, FuelType.AI95, FuelType.DT],
21         5: [FuelType.AI92, FuelType.AI95, FuelType.DT],
22         6: [FuelType.AI92, FuelType.AI95, FuelType.AI98,
23            FuelType.DT],
24         7: [FuelType.AI95, FuelType.DT],
25         8: [FuelType.AI95, FuelType.DT]
26     }
27
28     # Назначаем цистерны
29     for pump_id, fuels in pump_config.items():
30         nozzles = []
31         for fuel in fuels:
32             if fuel == FuelType.AI95:
33                 # Колонки 1-4 → цистерна 1, 5-8 → цистерна 2
34                 tank_id = 1 if pump_id <= 4 else 2
35             else:
36                 tank_id = 1
37             tank = tank_map[(fuel, tank_id)]
38             nozzles.append(Nozzle(fuel, tank))
39         self.pumps.append(Pump(pump_id, nozzles))

```

Рисунок 9 – Листинг программы для init default setup

- Создает 5 цистерн разных типов;
- Настраивает 8 заправочных колонок;
- Распределяет цистерны по колонкам (особенно для АИ-95).

Важный блок для моделирования физического устройства: как именно связаны колонки и цистерны.

2.11 Сохранение и загрузка состояния

Сохраняет все данные АЗС, а также восстанавливает их при запуске. На рисунке 10 представлен код программы.

```

1 def save_to_files(self):
2     data = {
3         "tanks": [t.to_dict() for t in self.tanks],
4         "operations": [op.to_dict() for op in self.
5             operations],
6         "stats": self.stats,
7         "emergency_mode": self.emergency_mode
8     }
9     with open("gas_station_data.json", "w", encoding="utf-8"
10 ) as f:
11         json.dump(data, f, ensure_ascii=False, indent=2)
12
13 def load_from_files(self):
14     if not os.path.exists("gas_station_data.json"):
15         return
16     try:
17         with open("gas_station_data.json", "r", encoding="
18             utf-8") as f:
19             data = json.load(f)
20             self.tanks = [Tank.from_dict(t) for t in data["tanks
21                 "]]
22             self.operations = [Operation.from_dict(op) for op in
23                 data["operations"]]
24             self.stats = data["stats"]
25             self.emergency_mode = data.get("emergency_mode",
26                 False)
27             # Пересоздаем колонки с новыми ссылками на цистерны
28             tank_map = {(t.fuel_type, t.tank_id): t for t in
29                 self.tanks}
30             ...
31             # далее( пересоздается self.pumps аналогично
32             _init_default_setup)
33     except Exception as e:
34         print(f"Ошибка загрузки данных: {e}")

```

Рисунок 10 – Листинг программы для сохранения и загрузка состояния

- save to files - сохраняет все данные АЗС (состояние цистерн, операции, статистику) в JSON-файл;

- load from files - загружает данные из JSON-файла при запуске.

Надёжный способ продолжать работу с данными после перезапуска программы и обработка ошибок предотвращает падение при проблемах с файлом.

2.12 Вспомогательные методы

Централизованное логирование упрощает отслеживание событий. Ограничение длины истории экономит память и делает вывод короче и понятнее. На рисунке 11 представлен код программы.

```

1 def log_operation(self, op_type: str, details: str):
2     op = Operation(op_type, details)
3     self.operations.append(op)
4     if len(self.operations) > 100: # ограничим историю
5         self.operations.pop(0)
6
7 def get_disabled_tanks(self) -> List[Tank]:
8     return [t for t in self.tanks if not t.enabled]

```

Рисунок 11 – Листинг программы для вспомогательных методов

- log operation: добавляет запись в историю и держит историю в пределах 100 различных записей.

- get disabled tanks: возвращает список отключённых цистерн.

Полезные утилиты для ведения истории и проверки состояния.

2.13 Метод serve customer

Обслуживание клиента / касса, сокращённая цитата, ключевые части. На рисунке 12 представлен код программы.

```

1 def serve_customer(self):
2     if self.emergency_mode:
3         print("АЗС заблокирована...")
4         return
5
6     available_pumps = [p for p in self.pumps if p.
7 get_available_fuels()]
8     # показывает список колонок, выбирается колонка
9     # выбирается топливо, вводится литраж
10    price_per_liter = self.FUEL_PRICES[nozzle.fuel_type]
11    total = liters * price_per_liter
12
13    confirm = input("Подтвердить оплату? (y/n)\n> ").strip().
14    lower()
15    if confirm != 'yes':
16        print("Операция отменена.")
17        return
18
19    try:
20        nozzle.tank.withdraw(int(liters))
21        self.stats["cars_served"] += 1
22        self.stats["fuel_sold"][nozzle.fuel_type] += int(
23        liters)
24        self.stats["fuel_income"][nozzle.fuel_type] += total
25        self.stats["total_income"] += total
26        self.log_operation("Продажа", f"{int(liters)} л {
27        nozzle.fuel_type} на колонке {pump_id}, сумма: {total:.2f}
28        P")
29        print("Операция выполнена успешно.")
30    except ValueError as e:
31        print(f"ОШИБКА:\n{e}")

```

Рисунок 12 – Листинг программы для метода serve customer

- Ведет весь диалог продажи топлива: выбор колонки, выбор топлива, ввод литров, расчет суммы, подтверждение, списание топлива из цистерны, обновление статистики и логирование действий над заправочной станцией;

- Обработывает ошибки (недостаточно топлива, отключенная цистерна).

Самая важная функция для повседневной работы станции; реализует пользовательский поток продажи с базовыми проверками.

2.14 check tanks

Печатает список всех цистерн с текущими уровнями и состоянием. На рисунке 13 представлен код программы.

```
1 def check_tanks(self):
2     for i, tank in enumerate(self.tanks, 1):
3         print(f"{i}) {tank}")
```

Рисунок 13 – Листинг программы для метода check tanks

Нужный простой дисплей статуса цистерн. Быстрый контроль состояния, удобный для оператора.

2.15 refill tank

Позволяет оператору добавить топливо в выбранную цистерну (с проверкой на переполнение). Логирует операцию. На рисунке 14 представлен код программы.

```
1 def refill_tank(self):
2     if self.emergency_mode:
3         print("Нельзя пополнять топливо в аварийном режиме!")
4         return
5
6     # меню выбора цистерны и ввод литров
7     tank.refill(liters)
8     self.log_operation("Пополнение", f"Цистерна {tank.fuel_type} №{tank.tank_id}, +{liters} л")
```

Рисунок 14 – Листинг программы для метода refill tank

Операция пополнения, с защитой от переполнения. Ведется учёт пополнений.

2.16 show balance

Печатает ключевые показатели: число обслуженных машин, общий доход, сколько каждого топлива продано и сколько принесло денег. На рисунке 15 представлен код программы.

```
1 def show_balance(self):
2     print(f"Обслужено автомобилей: {self.stats['cars_served']}")
3     print(f"Общий доход: {self.stats['total_income']:, .2f} ₺")
4     for ft in [FuelType.AI92, FuelType.AI95, FuelType.AI98, FuelType.DT]:
5         liters = self.stats["fuel_sold"][ft]
6         income = self.stats["fuel_income"][ft]
7         print(f"{ft:<6} - {liters} л ({income:,.2f} ₺)")
```

Рисунок 15 – Листинг программы для метода show balance

Быстрое подведение итогов работы смены или дня.

2.17 transfer fuel

Позволяет перемещать топливо между цистернами того же типа (например, перенести часть АИ-95 из одной цистерны в другую). Проверяет, что есть место в приёмнике и что источник имеет достаточно топлива. На рисунке 16 представлен код программы.

```

1 def transfer_fuel(self):
2     if self.emergency_mode:
3         print("Нельзя перекачивать топливо в аварийном режиме!")
4     )
5     return
6     sources = [t for t in self.tanks if t.enabled]
7     # выбор источника, выбор приемника того же типа топлива
8     source.withdraw(liters)
9     target.refill(liters)
10    self.log_operation("Перекачка", f"{liters} л {source.fuel_type} из №{source.tank_id} в №{target.tank_id}")

```

Рисунок 16 – Листинг программы для метода transfer fuel

- Практичная функция для балансировки уровней и подготовки цистерн;
- Помогает избежать ситуаций, когда одна цистерна пуста, а другая переполнена.

Реалистичная операция обслуживания склада топлива с проверками безопасности.

2.18 manage tanks

Дает оператору возможность вручную включать/отключать цистерны. При включении проверяется, не ниже ли уровень минимального порога. На рисунке 17 представлен код программы.

```

1 def manage_tanks(self):
2     if self.emergency_mode:
3         print("Управление цистернами недоступно в аварийном режиме!")
4     return
5
6     # варианты: включить цистерну если( не ниже минимума), или отключить
7     if action == "1":
8         disabled = [t for t in self.tanks if not t.enabled]
9         # попытка включить
10    elif action == "2":
11        enabled = [t for t in self.tanks if t.enabled]
12        # отключение

```

Рисунок 17 – Листинг программы для метода manage tanks

- Позволяет контролировать какие цистерны используются (например, при ремонте или планировании);
 - Предотвращает включение пустых цистерн.
- Удобный интерфейс администрирования состояния цистерн.

2.19 show pumps

Печатает для каждой колонки, какие сопла есть и доступны ли они в данный момент. На рисунке 18 представлен код программы.

```

1 def show_pumps(self):
2     for pump in self.pumps:
3         print(f"Колонка {pump.pump_id}:")
4         for nozzle in pump.get_all_fuels():
5             status = "РАБОТАЕТ" if nozzle.is_available() else
              "НЕ РАБОТАЕТ"
6         print(f" - {nozzle.fuel_type} цистерна( {nozzle.
              get_tank_info()}) → {status}")

```

Рисунок 18 – Листинг программы для метода show pumps

Наглядный статус колонок для оператора.

2.20 show warning

При запуске меню показывает, если есть отключённые цистерны и почему (низкий уровень или отключили вручную). На рисунке ?? представлен код программы.

```

1 def show_warning(self):
2     disabled = self.get_disabled_tanks()
3     if disabled:
4         print("ВНИМАНИЕ!")
5         print("Обнаружены отключённые цистерны:")
6         for t in disabled:
7             reason = "низкий уровень топлива" if t.is_low()
              else "вручную"
8         print(f" - {t.fuel_type} №{t.tank_id} ({reason})")
9     )

```

Рисунок 19 – Листинг программы для метода show warning

Удобное напоминание оператору, повышает внимательность.

2.21 run

Загружает данные при старте, отображает меню в цикле, читает выбор пользователя и вызывает соответствующие методы. На выходе сохраняет данные и завершает программу. На рисунке 20 представлен код программы.

```

1 def run(self):
2     self.load_from_files()
3     while True:
4         os.system('cls' if os.name == 'nt' else 'clear')
5         # печать заголовка и меню
6         choice = input("> ").strip()
7         if choice == "1":
8             self.serve_customer()
9         ...
10        elif choice == "0":
11            self.save_to_files()
12            print("Данные сохранены. До свидания!")
13            break

```

Рисунок 20 – Листинг программы для метода run

Типичный простой консольный интерфейс, удобный для демонстрации и для использования вручную.

2.22 start program

Запускает программу, если файл выполнен напрямую (а не импортирован как модуль). На рисунке 21 представлен код программы.

```
1 if __name__ == "__main__":  
2     station = GasStation()  
3     station.run()
```

Рисунок 21 – Листинг программы для метода start program

Вся программа запускается из этого блока.

3 Тестирование

Тестирование системы управления автозаправочной станцией проводилось с целью проверки корректности работы всех функциональных компонентов и обеспечения надежности системы. Были выполнены следующие виды тестирования:

- Попытка заправиться от отключенной и корректно функционирующей цистерны;
- Тестирование производительности: Оценка скорости обработки запросов и отклика системы при различных нагрузках.
- Сохранение и восстановление состояния;
- Ручное включение цистерн после включения;
- Пополнение цистерн с проверкой на переполнение.

Все запуски прошли успешно, система работает корректно и стабильно.

4 Заключение

Разработанный полностью работоспособный прототип системы управления автозаправочной станцией на языке Python продемонстрировал свою эффективность и надежность в ходе тестирования. Система успешно реализует основные функции, необходимые для управления АЗС, включая обслуживание клиентов, учет топлива, управление цистернами и ведение истории операций.

Все поставленные цели были достигнуты, и система соответствует требованиям технического задания. В ходе разработки были применены принципы программирования, что обеспечило модульность, удобство использования и безопасность системы.